

Classification bayésienne supervisée de pixels : analyse des espaces colorimétriques RGB, HSV et Y'CbCr

1st Santiago Florido Gomez
Ingénieur Degree Programme STIC
ENSTA Paris
Paris, France
santiago.florido@ensta.fr

2nd Javier Andres Tarazona Jimenez
Ingénieur Degree Programme STIC
ENSTA Paris
Paris, France
javier-andres.tarazona@ensta.fr

3rd Jose daniel Chacon Gomez
Ingénieur Degree Programme STIC
ENSTA Paris
Paris, France
jose-daniel.chacon@ensta.fr

Abstract—Ce travail présente un cadre de classification bayésienne supervisée appliqué à la segmentation de pixels en imagerie couleur. L'article formalise les critères de décision Maximum A Posteriori (MAP) et Maximum Vraisemblance (ML), puis étudie le rôle du choix de l'espace d'observation dans la séparabilité statistique des classes. Trois représentations colorimétriques sont analysées : RGB, HSV et Y'CbCr. Nous décrivons leurs transformations, leurs propriétés pertinentes pour l'inférence probabiliste, et l'extraction opérationnelle de leurs composantes au moyen d'une implémentation Python/OpenCV. Les visualisations et canaux exportés (R, G, B, H, S, V, Y', Cb, Cr) fournissent une base reproductible pour la construction de modèles bayésiens de classification de pixels et la comparaison de performances entre espaces colorimétriques.

Index Terms—Classification bayésienne, Maximum A Posteriori, Maximum Vraisemblance, segmentation de pixels, RGB, HSV, Y'CbCr, OpenCV.

I. PRINCIPES DE LA CLASSIFICATION BAYÉSIENNE

Les méthodes d'apprentissage bayésien partent d'un ensemble de données d'observation noté X et d'un ensemble de classes c . Dans ce cadre, une observation est modélisée sous forme vectorielle, et l'on peut définir une probabilité a priori d'observer le vecteur X , notée $P(X)$, ainsi qu'une probabilité a priori d'appartenance à une classe c , notée $P(c)$, comme indiqué dans (1) et (2). On peut également exprimer la probabilité conjointe $P(X, c)$, donnée en (3).

$$P(X) \quad (1)$$

$$P(c) \quad (2)$$

$$P(X, c) \quad (3)$$

En utilisant le concept de probabilité conditionnelle, il est possible d'exprimer la probabilité d'observer les données X sachant que l'observation appartient à la classe c . Cette probabilité conditionnelle, notée $P(X | c)$, est aussi appelée vraisemblance (conditionnelle), comme en (4).

$$P(X | c) \quad (4)$$

La probabilité conjointe peut alors s'écrire comme le produit de la vraisemblance et de l'a priori de classe, ce qui donne (5).

$$P(X, c) = P(X | c) P(c) \quad (5)$$

Enfin, en appliquant le théorème de Bayes, on obtient une probabilité a posteriori $P(c | X)$, qui exprime la probabilité qu'une observation X appartienne à une classe spécifique c dans le domaine du système. Cette relation est donnée par (6).

$$P(c | X) = \frac{P(X | c) P(c)}{P(X)} \quad (6)$$

L'apprentissage bayésien consiste donc en la détermination empirique des lois de probabilité, référée à partir d'un ensemble de données statistiques, réalisée sur un ensemble de données d'apprentissage étiquetées $\{X_i, c(X_i)\}$. La détermination de la classe c donne lieu à deux formes de sélection en fonction du critère d'optimisation employé.

A. Maximum A Posteriori (MAP)

On se focalise sur un critère de maximisation de la probabilité a posteriori, de sorte qu'à partir d'une observation X , on cherche la sélection de la classe c la plus probable [1], suivant l'expression (7).

$$c_{\text{MAP}}^*(X) = \arg \max_{c \in C} P(c | X). \quad (7)$$

En utilisant des logarithmes (utile numériquement et pour l'analyse), le critère MAP peut s'écrire comme indiqué en (8).

$$c_{\text{MAP}}^*(X) = \arg \max_{c \in C} (\log P(X | c) + \log P(c)). \quad (8)$$

La probabilité a priori peut être estimée à partir de la base de données d'entraînement par la relation (9), où N_c désigne le nombre d'exemples appartenant à la classe c , défini en (10).

$$P(c) = \frac{N_c}{N}. \quad (9)$$

$$N_c = |\{i : c_i = c\}|. \quad (10)$$

De son côté, pour la vraisemblance, on ajuste les données observées à un modèle $P(X | c)$ pour chaque classe (par exemple : histogramme, Naive Bayes, ou modèle gaussien).

B. Maximum Vraisemblance (ML)

De son côté, le critère de ML (Maximum Likelihood) cherche la sélection de la classe qui rend la donnée X d'intérêt pour la classification la plus vraisemblable, au moyen de l'expression de maximisation (11).

$$c_{ML}^*(X) = \arg \max_{c \in C} P(X | c). \quad (11)$$

Il suit ainsi un principe de recherche de la classe pour laquelle la donnée observée X [2].

Dans la représentation logarithmique de la maximisation, il devient évident que l'on conserve l'effet de la vraisemblance, et qu'à la différence du MAP il n'y a pas d'influence de la distribution a priori de la classe, ce qui conduit à (12).

$$c_{ML}^*(X) = \arg \max_{c \in C} \log P(X | c). \quad (12)$$

Cela rend compte du fait que la principale différence entre ML et MAP est le déplacement des limites de sélection à cause de la nature de la fonction a priori pour chaque classe. Ainsi, dans le cas d'une classification à deux classes, ML utilisera le seuil 0 défini en (13).

$$\log P(X | c_1) - \log P(X | c_2) > 0. \quad (13)$$

Et pour MAP, on obtient le critère (14).

$$\log P(X | c_1) - \log P(X | c_2) > \log \frac{P(c_2)}{P(c_1)}. \quad (14)$$

Cela implique que si $P(c_1)$ est grande, alors $\log \frac{P(c_2)}{P(c_1)}$ est négatif, et MAP a besoin de moins d'évidence (en termes de likelihood) pour choisir c_1 . On conclut donc que MAP, à la différence de ML, considère dans la classification le fait que certaines classes, par leur propre dynamique et nature, sont plus probables que d'autres, ce qui peut produire un effet en classification qui n'est pas désiré lorsque les classes sont, par exemple, déséquilibrées.

C. Application 'a la classification bayésienne des pixels

Dans le contexte de la classification supervisée d'images par classification bayésienne, la définition de l'espace d'observation correspond à un choix méthodologique central, puisqu'elle conditionne la séparabilité statistique entre les classes et, par conséquent, la qualité du modèle probabiliste appris et implémenté. Comme première approche, il est proposé d'analyser la représentation de chaque pixel en termes colorimétriques, et d'étudier différents espaces de représentation, spécifiquement RGB, HSV et Y'CbCr.

1) *Espace RGB*: Du point de vue colorimétrique, un espace RGB est une représentation du signal coloré avec des composantes associées aux trois primaires, ce qui permet de représenter tout stimulus visible au moyen de valeurs tristimulus. Il peut donc être compris comme une base tridimensionnelle. RGB représente une famille d'espaces qui n'est complètement spécifiée que lorsque les chromaticités des primaires, le blanc de référence et la fonction de transfert entre le signal linéaire et le signal codé sont fixés.

Dans cette représentation, chaque pixel est modélisé sous la forme d'un vecteur tridimensionnel, comme indiqué dans l'équation (15) :

$$\mathbf{x}_{RGB}(u, v) = \begin{bmatrix} R(u, v) \\ G(u, v) \\ B(u, v) \end{bmatrix}, \quad (15)$$

où R , G et B décrivent les contributions associées aux primaires rouge, verte et bleue. Dans l'implémentation considérée, l'extraction des composantes RGB ne constitue pas une transformation colorimétrique additionnelle, mais une simple permutation de canaux à partir de l'image BGR chargée par OpenCV. En effet, si le signal d'entrée est représenté comme dans l'équation (16) :

$$\mathbf{x}_{BGR}(u, v) = \begin{bmatrix} B(u, v) \\ G(u, v) \\ R(u, v) \end{bmatrix}, \quad (16)$$

alors l'opération `cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)` [5] est équivalente à la transformation exprimée dans l'équation (17) :

$$\mathbf{x}_{RGB} = P \mathbf{x}_{BGR}, \quad P = \begin{bmatrix} 0 & 0 & 1 \\ 0 & 1 & 0 \\ 1 & 0 & 0 \end{bmatrix}. \quad (17)$$

Par la suite, `cv2.split` permet d'obtenir trois champs scalaires monobandes, correspondant aux projections du vecteur couleur sur la base canonique RGB. En conséquence, les images exportées sous les noms `R.png`, `G.png` et `B.png` ne représentent pas de nouvelles couleurs, mais des cartes d'intensité de chaque canal.



Fig. 1. Composantes de l'espace RGB (image originale, R, G, B).

2) *Espace HSV*: Pour sa part, l'espace HSV constitue une reparamétrisation non linéaire du cube RGB en fonction de la teinte, de la saturation et de la valeur, et il part du modèle hexaédrique dans lequel le cube RGB est projeté le long de la diagonale achromatique. Dans ce cadre, la teinte est employée comme une description de la position angulaire sur le contour chromatique, et la saturation comme une quantification de l'éloignement par rapport à l'axe des gris. Smith montre en

outre que V peut être interprété comme la hauteur de la barre chromatique dominante [4].

Dans le code implémenté, cette transformation est réalisée au moyen d'OpenCV avec `cv2.COLOR_BGR2HSV` [5]. Les composantes BGR sont d'abord converties en virgule flottante et normalisées dans l'intervalle $[0, 1]$, après quoi les équations du modèle HSV sont appliquées. La composante valeur est définie par l'équation (18) :

$$V = \max(R, G, B), \quad (18)$$

tandis que la saturation est donnée par l'équation (19) :

$$S = \begin{cases} \frac{V - \min(R, G, B)}{V}, & V \neq 0, \\ 0, & V = 0, \end{cases} \quad (19)$$

et la teinte par l'équation (20) :

$$H = \begin{cases} \frac{60(G - B)}{V - \min(R, G, B)}, & V = R, \\ 120 + \frac{60(B - R)}{V - \min(R, G, B)}, & V = G, \\ 240 + \frac{60(R - G)}{V - \min(R, G, B)}, & V = B, \\ 0, & R = G = B. \end{cases} \quad (20)$$

Si $H < 0$, OpenCV ajoute 360 degrés afin de replacer la teinte dans l'intervalle angulaire standard [5].

Néanmoins, l'image de sortie reste codée sur 8 bits et ne stocke pas la teinte en degrés physiques, mais dans une version quantifiée. La documentation indique que, pour les images sur 8 bits, l'échelle de sortie est donnée par l'équation (21) :

$$V \leftarrow 255V, \quad S \leftarrow 255S, \quad H \leftarrow \frac{H}{2}, \quad (21)$$

de sorte que, en pratique, le canal exporté par `H.png` appartient à l'intervalle discret indiqué dans l'équation (22) :

$$H_{\text{OpenCV}} \in \{0, \dots, 179\}, \quad (22)$$

tandis que S et V sont codés dans $\{0, \dots, 255\}$ [5]. C'est pour cette raison qu'il n'est pas possible d'interpréter directement la teinte stockée comme un angle en degrés, mais plutôt comme une résolution discrétisée dans laquelle chacun des niveaux de teinte correspond approximativement à deux degrés.

HSV présente un intérêt particulier dans la mesure où il permet le découplage des attributs chromatiques perceptifs, ce qui peut être utile pour faciliter la discrimination des classes qui peuvent être davantage différenciées par la teinte que par l'intensité absolue.



Fig. 2. Composantes de l'espace HSV (image originale, H, S, V).

3) *Espace* (Y', Cr, Cb) : Se obtiennent les composantes $Y'CbCr$ dans l'ordre (Y', Cr, Cb) , puisque la transformation appliquée est `cv2.COLOR_BGR2YCrCb`, où OpenCV organise en interne la sortie comme la luminance Y' , la chrominance rouge-différence Cr et la chrominance bleue-différence Cb . Le code compense correctement ce détail en dépaquetant la sortie sous la forme `y_channel`, `cr_channel`, `cb_channel`, puis en la réordonnant lors de l'enregistrement de `Cb.png` et `Cr.png`.

La documentation d'OpenCV décrit cette conversion comme $RGB \leftrightarrow YCrCb \text{ JPEG (or YCC)}$ [6] et la définit par les équations (23) à (25) :

$$Y' = 0.299R + 0.587G + 0.114B, \quad (23)$$

$$Cr = 0.713(R - Y') + \delta, \quad (24)$$

$$Cb = 0.564(B - Y') + \delta, \quad (25)$$

où

$$\delta = \begin{cases} 128, & \text{pour les images sur 8 bits,} \\ 32768, & \text{pour les images sur 16 bits,} \\ 0.5, & \text{en virgule flottante.} \end{cases} \quad (26)$$

La transformation inverse est définie par les équations (27) à (29) :

$$R = Y' + 1.403(Cr - \delta), \quad (27)$$

$$G = Y' - 0.714(Cr - \delta) - 0.344(Cb - \delta), \quad (28)$$

$$B = Y' + 1.773(Cb - \delta). \quad (29)$$

Ces équations montrent que la représentation est essentiellement une transformation affine de l'espace RGB, suivie d'un recentrage des chrominances.

Le principal avantage de cette transformation réside dans le découplage partiel entre l'information de luminance apparente et l'information chromatique différentielle, ce qui peut conduire à des distributions de couleur plus compactes dans le sous-espace (Cr, Cb) . En outre, contrairement au cas de HSV, elle n'introduit pas une coordonnée angulaire singulière, mais fonctionne essentiellement comme une transformation linéaire des canaux d'entrée.

L'utilisation exclusive de la couleur renvoie à l'implémentation d'un vecteur de faible dimension comme base d'observation, tel que défini dans l'équation (30) :



Fig. 3. Composantes de l'espace Y'CbCr (image originale, Y', Cb, Cr).

$$\mathbf{x} \in \left\{ \begin{bmatrix} R \\ G \\ B \end{bmatrix}, \begin{bmatrix} H \\ S \\ V \end{bmatrix}, \begin{bmatrix} Y \\ Cb \\ Cr \end{bmatrix} \right\}. \quad (30)$$

4) *Couleurs comme espace d'observation*: Le principal problème est que, dans une telle base, il n'est pas possible de décrire de manière suffisante la classe réelle du pixel, en raison des limitations liées à la forte dépendance à l'illumination lorsqu'une représentation fondée uniquement sur la couleur est employée. Ainsi, la couleur identifiée peut varier sous l'effet des ombres, de l'intensité lumineuse, des reflets, du déséquilibre des blancs et de la réponse du capteur, ce qui rend très probable qu'un même objet, ou des objets appartenant à une même classe, occupent des régions différentes de l'espace colorimétrique selon les conditions d'acquisition de l'image.

De même, deux classes distinctes peuvent être associées à des couleurs similaires, ou bien une même classe peut présenter une large variabilité chromatique. En termes statistiques, cela peut produire un chevauchement entre les distributions.

La description uniquement fondée sur la couleur des pixels peut également être liée à une absence d'information spatiale, étant donné que chaque pixel ne contient pas d'information sur son voisinage dans l'image. Par conséquent, une représentation locale basée uniquement sur la couleur peut conduire à des résultats bruités ou à des pixels mal classés.

Enfin, une représentation de ce type peut être insuffisante, puisque des régions ayant des significations différentes peuvent partager une même couleur tout en différant par leur texture, leurs contours, leur orientation ou leur forme.

Compte tenu des limitations d'un espace uniquement colorimétrique, il est proposé de définir un espace d'observation efficace combinant information photométrique et information géométrique. Il est ainsi proposé de partir d'une description construite à partir des composantes de couleur déjà référencées, mais complétée par des caractéristiques différentielles locales. Parmi celles-ci, on peut considérer des éléments comme le gradient, défini par les équations (31) et (32),

$$\nabla I = \begin{bmatrix} I_x \\ I_y \end{bmatrix}, \quad (31)$$

$$I_g = \|\nabla I\| = \sqrt{I_x^2 + I_y^2}, \quad (32)$$

qui mesure l'intensité des variations locales et permet de décrire les contours et les transitions. Il est également possible de prendre en considération des caractéristiques de second ordre, comme celles obtenues à partir de la matrice hessienne donnée par l'équation (33),

$$H_I = \begin{bmatrix} I_{xx} & I_{xy} \\ I_{xy} & I_{yy} \end{bmatrix}, \quad (33)$$

dont les valeurs propres décrivent les courbures principales de l'image. Si l'on note ces valeurs propres par λ_I et Λ_I , avec $\lambda_I \leq \Lambda_I$, celles-ci peuvent s'écrire comme

$$\lambda_I = \beta_I - \alpha_I, \quad \Lambda_I = \beta_I + \alpha_I, \quad (34)$$

où

$$\beta_I = \frac{1}{2}(I_{xx} + I_{yy}), \quad \alpha_I = \frac{1}{2}\sqrt{(I_{xx} - I_{yy})^2 + 4I_{xy}^2}. \quad (35)$$

À partir de cette matrice, on peut extraire le laplacien, défini par l'équation (36),

$$\Delta I = I_{xx} + I_{yy} = \lambda_I + \Lambda_I, \quad (36)$$

la norme de Frobenius, définie par l'équation (37),

$$\|H_I\|_F = \sqrt{I_{xx}^2 + I_{yy}^2 + 2I_{xy}^2} = \sqrt{\lambda_I^2 + \Lambda_I^2}, \quad (37)$$

et le déterminant, donné par l'équation (38),

$$\det(H_I) = I_{xx}I_{yy} - I_{xy}^2 = \lambda_I\Lambda_I. \quad (38)$$

Ces quantités sont utiles pour l'identification efficace de zones planes, de rampes, de structures elliptiques, de crêtes, de *blobs* et d'autres motifs géométriques. En effet, lorsque

$$\|\nabla I\| \simeq 0, \quad \|H_I\|_F \simeq 0, \quad (39)$$

on peut considérer que l'on est en présence d'une zone plane ou *plateau*. En revanche, lorsque

$$\|\nabla I\| \gg 0, \quad \|H_I\|_F \simeq 0, \quad (40)$$

la structure locale correspond à une rampe ou *rampe*. Enfin, lorsque

$$\|H_I\|_F \gg 0, \quad (41)$$

on a une structure gouvernée par la courbure, dont la nature dépend du signe du produit $\lambda_I\Lambda_I$: si

$$\lambda_I\Lambda_I > 0, \quad (42)$$

la structure est de type elliptique ou *blob*, tandis que si

$$\lambda_I\Lambda_I < 0, \quad (43)$$

la configuration est de type *selle* ou, dans certains cas, tubulaire ou de crête.

Par ailleurs, il est important de noter que ces dérivées ne doivent pas être comprises uniquement comme des quantités numériques isolées, mais comme une véritable représentation locale du voisinage du pixel. En effet, le comportement local de l'image dans un voisinage de (x_0, y_0) peut être approché au moyen d'un développement de Taylor du second ordre,

$$I(x_0 + \varepsilon, y_0 + \eta) \approx I(x_0, y_0) + \nabla I(x_0, y_0) \begin{bmatrix} \varepsilon \\ \eta \end{bmatrix} + \frac{1}{2} \begin{bmatrix} \varepsilon & \eta \end{bmatrix} H_I(x_0, y_0) \begin{bmatrix} \varepsilon \\ \eta \end{bmatrix}, \quad I_s = P_s(I), \quad (54)$$

et, dans le cas linéaire, comme une convolution,

$$I_s = I \star h_s, \quad (55)$$

sous des hypothèses d'invariance et de propriété de semi-groupe, l'opérateur de changement d'échelle doit coïncider avec un noyau gaussien, c'est-à-dire

$$h_s = G_{\sqrt{2}s}. \quad (56)$$

C'est pourquoi les espaces d'échelle linéaires sont identifiés à des espaces d'échelle gaussiens. En conséquence, les dérivées à l'échelle s s'obtiennent par convolution avec des dérivées de gaussienne,

$$\left(\frac{\partial^{i+j} I}{\partial x^i \partial y^j} \right)_s = I \star G_{\sqrt{2}s}^{(i,j)}. \quad (57)$$

Par conséquent, l'incorporation de descripteurs ou de caractéristiques multi-échelles ne doit pas être comprise uniquement comme une stratégie consistant à "essayer plusieurs tailles", mais comme la manière correcte de définir une représentation locale cohérente.

Compte tenu de cela, et dans la recherche d'un descripteur qui conserve l'information chromatique, qui ajoute une information géométrique locale, qui introduise une structure multi-échelle, et qui conserve en outre une représentation relativement compacte, il est proposé d'utiliser le vecteur défini par l'équation (58) :

$$\mathbf{x}(p) = \begin{bmatrix} Cb(p) \\ Cr(p) \\ \|\nabla I\|_{s_1}(p) \\ \Delta I_{s_1}(p) \\ \det(H_I)_{s_1}(p) \\ \|\nabla I\|_{s_2}(p) \\ \Delta I_{s_2}(p) \\ \det(H_I)_{s_2}(p) \end{bmatrix}. \quad (58)$$

où les dérivées à l'échelle s sont obtenues par convolution avec des dérivées de gaussienne.

Ce vecteur combine une information photométrique et une information géométrique, et se révèle plus approprié pour la segmentation bayésienne de pixels qu'une représentation purement colorimétrique.

II. CLASSIFICATION DE CARACTÉRISTIQUES LOCALES

Dans la suite du TP, nous avons choisi de traiter le problème *Visages* de la base d'images Essex. L'objectif général est d'isoler les pixels de la peau du reste de l'image, afin de localiser précisément le visage sur l'image à partir d'une classification de caractéristiques locales.

ce qui montre que l'intensité, le gradient et la hessienne constituent une représentation compacte et géométriquement interprétable de l'environnement local.

De plus, le gradient ne mesure pas seulement l'intensité des variations locales, mais définit également la direction de pente maximale. Si l'on considère un vecteur unitaire v , la dérivée directionnelle est donnée par

$$\frac{\partial I}{\partial v} = \nabla I \cdot v. \quad (45)$$

En introduisant le repère local (g, t) , avec

$$g = \frac{\nabla I}{\|\nabla I\|}, \quad t = g^\perp, \quad (46)$$

on obtient

$$I_g = \|\nabla I\|, \quad I_t = 0. \quad (47)$$

De cette manière, g correspond à la direction principale de variation et t à la direction de l'isophote ou ligne de niveau locale.

Dans ce même repère local, les dérivées secondes dans la direction du gradient et dans la direction tangentielle peuvent s'exprimer comme

$$I_{gg} = g^T H_I g = \frac{I_{xx}I_x^2 + 2I_{xy}I_xI_y + I_{yy}I_y^2}{I_x^2 + I_y^2}, \quad (48)$$

$$I_{tt} = \frac{I_{xx}I_y^2 - 2I_{xy}I_xI_y + I_{yy}I_x^2}{I_x^2 + I_y^2}. \quad (49)$$

De plus, ces quantités vérifient la relation

$$I_{gg} + I_{tt} = I_{xx} + I_{yy} = \Delta I. \quad (50)$$

Cela permet d'interpréter le laplacien comme une quantité invariante par rotation. Si, en outre, la courbure de l'isophote est faible, c'est-à-dire si

$$I_{tt} \approx 0, \quad (51)$$

alors on a approximativement

$$I_{gg} \approx \Delta I, \quad (52)$$

et la condition de contour peut être approchée par

$$\Delta I = 0. \quad (53)$$

En tenant également compte du fait que la géométrie locale étudiée jusqu'à présent dans les propositions réalisées est subordonnée à une échelle particulière, il est proposé de travailler avec des descripteurs multi-échelles. En considérant l'échelle comme dans l'équation (54),

A. Classification bayésienne des pixels

La classification bayésienne des pixels consiste à attribuer à chaque pixel de l'image une classe en fonction d'un vecteur d'observation construit à partir de caractéristiques locales, typiquement les composantes de couleur ou d'autres descripteurs calculés autour du pixel. Dans le cadre de ce TP, l'idée est de modéliser probabilistiquement chaque classe à partir d'exemples annotés, puis de décider pour chaque pixel la classe la plus probable au sens bayésien.

Conformément à la section 3.1 du sujet, on commence par visualiser les composantes pertinentes avec le script `Display_Components.py`, puis on sélectionne des exemples d'apprentissage pour apprendre un modèle avec `Bayes_Model_Training.py`. L'apprentissage est supervisé : l'utilisateur fournit, à l'aide de la souris et du clavier, des régions d'exemples appartenant à la classe positive et à la classe négative. Le modèle estimé permet ensuite de classer l'ensemble des pixels de l'image selon la probabilité associée à chaque classe.

Dans notre problème *Visages*, cette approche revient à distinguer principalement les pixels de peau des pixels de non-peau. La classe positive correspond donc aux zones du visage que l'on souhaite extraire, tandis que la classe négative regroupe l'arrière-plan, les cheveux, les vêtements et les autres parties de l'image. L'objectif est d'obtenir une segmentation suffisamment précise pour localiser le visage de manière fiable.

1) **Analyse des couleurs:** Afin d'évaluer la séparabilité visuelle entre la peau et le non-peau avant l'apprentissage, nous avons exécuté le script `Display_Components.py` sur l'image `img/Essex_Faces/94/asamma.19.jpg`, issue du jeu de données des visages. Les sorties ont été sauvegardées dans `outputs/components/asamma.19/rgb`, `outputs/components/asamma.19/hsv` et `outputs/components/asamma.19/ycrcb` (voir Figure 4). Les fichiers `original.png` présents dans chacun de ces répertoires correspondent à la même image source, tandis que les figures de synthèse permettent de comparer les décompositions dans les trois espaces colorimétriques.

L'espace RGB fournit une lecture directe des intensités rouge, verte et bleue, mais il mélange l'information de chrominance et l'information d'illumination. Sur cette image, le fond vert est relativement bien distingué dans le canal *G*, et la peau ressort davantage dans le canal *R*. Néanmoins, les variations d'éclairage sur le front, le nez et les joues restent fortement présentes dans les trois canaux, ce qui risque de rendre le modèle moins stable d'une image à l'autre.

L'espace HSV sépare partiellement la couleur de la luminance via les composantes *H*, *S* et *V*. La composante *V* conserve la structure générale du visage, mais la composante *H* apparaît ici plus instable et bruitée, notamment dans les zones de faible saturation ou près des transitions abruptes. La composante *S* met en évidence certaines différences de peau et de fond, mais elle reste sensible aux reflets, aux ombres et aux zones sombres comme les cheveux.

L'espace YCrCb est le plus prometteur pour ce problème. La composante *Y* capture la luminance et donc la structure générale de la scène, tandis que *Cr* et *Cb* isolent mieux l'information chromatique liée à la peau. Dans les résultats obtenus, le visage reste relativement cohérent dans les composantes *Cr* et *Cb*, alors que l'arrière-plan et les cheveux se distinguent plus nettement. Cette décorrélation entre luminance et chrominance est favorable à l'apprentissage d'un modèle bayésien plus robuste pour la détection de la peau.

En conséquence, YCrCb constitue le meilleur point de départ pour construire un bon modèle de détection dans notre cas. Il existe cependant des opportunités claires pour enrichir l'espace d'observation avec des caractéristiques locales personnalisées. Une première piste consiste à combiner *Cb* et *Cr* avec une mesure de structure locale, par exemple le gradient de *Y*, afin d'introduire à la fois l'information de couleur de peau et l'information de contour. Une autre piste est d'utiliser des moyennes locales, des écarts-types sur fenêtre, ou des descripteurs de texture locale pour mieux distinguer la peau des cheveux, des ombres et de l'arrière-plan. Cette idée est cohérente avec l'extension `cocr_grad` déjà présente dans le code, qui combine *Cb*, *Cr* et $|\nabla Y|$ pour enrichir la représentation locale des pixels.

2) **Propres caractéristiques locales:** Pour aller au-delà de l'utilisation des seules composantes de couleur, nous avons ajouté une caractéristique locale de type *gradient et chrominance*, notée `cocr_grad`, et visualisée avec une extension du script `Display_Components.py`. Cette représentation combine les composantes *Cb* et *Cr*, qui conservent l'information de chrominance liée à la peau, avec la norme du gradient $|\nabla Y|$, calculée sur la luminance. Les sorties correspondantes ont été enregistrées dans `outputs/components/asamma.19/cocr_grad` (voir Figure 5).

Les composantes *Cb* et *Cr* gardent une bonne cohérence sur les zones de peau, comme dans l'espace YCrCb, mais l'ajout de $|\nabla Y|$ fait apparaître plus nettement les contours et transitions locales significatives : bord du visage, yeux, bouche, frontières entre la peau et les cheveux, ainsi qu'entre le visage et l'arrière-plan. Par rapport à YCrCb seul, cette combinaison introduit donc une information de structure locale sans quitter une représentation compacte à trois dimensions.

Cette caractéristique est plus favorable à un modèle bayésien de détection peau / non-peau, car elle aide à mieux distinguer les régions ambiguës où la couleur seule ne suffit pas, en particulier sur les frontières et dans les zones partiellement ombrées. Sa limite est que le gradient réagit aussi aux ombres, à certaines textures et aux accessoires. En pratique, cela signifie que les exemples négatifs utilisés pendant l'apprentissage doivent couvrir non seulement le fond, mais aussi les cheveux, les yeux et les zones sombres du visage, afin d'éviter qu'un contour marqué soit confondu avec une zone de peau.

3) **Pourquoi le descripteur (58) n'a-t-il pas été choisi ?:** Le descripteur défini par l'équation (58) est théoriquement plus riche et potentiellement plus discriminant que `cocr_grad`, car il combine l'information chromatique avec plusieurs

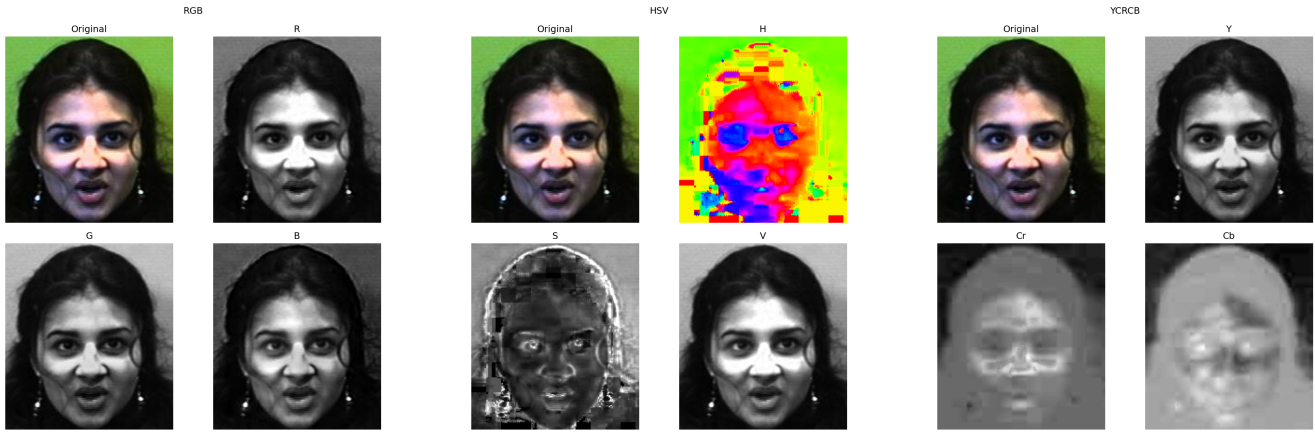


Fig. 4. Décompositions RGB, HSV et YCrCb de l'image `asamma.19.jpg`, générées avec `Display_Components.py`. Chaque figure contient l'image originale ainsi que les trois composantes de l'espace considéré.

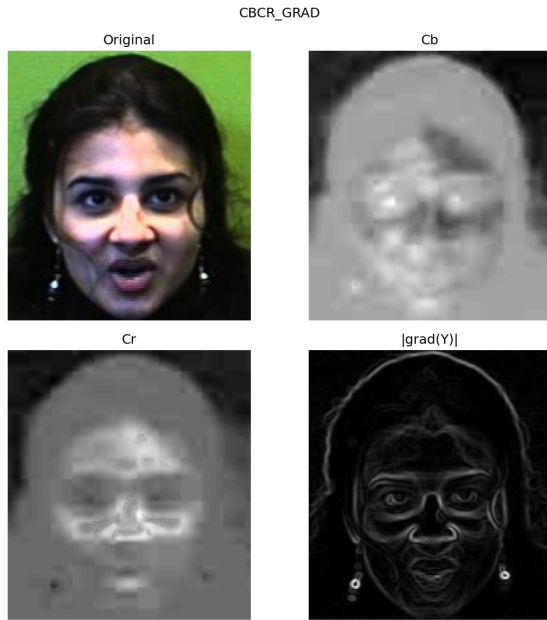


Fig. 5. Visualisation de la caractéristique locale `cocr_grad` sur l'image `asamma.19.jpg` : image originale, `Cb`, `Cr` et $|\nabla Y|$.

caractéristiques géométriques calculées à deux échelles. Il n'a cependant pas été implémenté ni évalué expérimentalement dans ce travail. Dans le cadre retenu ici, `cocr_grad` constitue donc un compromis plus simple et plus opérationnel : il n'utilise que trois variables (Cb , Cr , $|\nabla Y|$), alors que (58) comporte huit composantes. Cette augmentation de dimension complexifie l'apprentissage bayésien, en particulier pour des modèles simples comme `GaussianNB`, et accroît à la fois le besoin de normalisation, le risque de surapprentissage et la difficulté d'estimer correctement les paramètres à partir d'un nombre limité d'images annotées.

Par ailleurs, plusieurs variables dérivées présentes dans (58) sont corrélées, ce qui complique encore leur exploitation par

un modèle bayésien simple. Enfin, un descripteur plus riche ne corrige pas à lui seul une erreur de supervision : si l'annotation manuelle des RoI est imprécise ou incomplète, cette erreur humaine se propage au modèle quelle que soit la qualité théorique du vecteur d'observation. Le choix de `cocr_grad` ne doit donc pas être compris comme la preuve qu'il est supérieur à (58), mais comme la sélection d'un descripteur plus adapté au protocole expérimental effectivement mis en oeuvre dans ce rapport.

4) Modification du modèle d'entraînement: Le script `Bayes_Model_Training.py` a été adapté pour exploiter `cocr_grad` comme espace d'observation par défaut. L'interface accepte désormais une ou plusieurs images d'apprentissage en argument positionnel, ce qui permet soit de conserver le cas simple à une seule image, soit d'agréger des annotations provenant de plusieurs visages avant l'estimation du modèle bayésien. L'argument `-test-image` reste optionnel et, s'il n'est pas fourni, la première image d'apprentissage est utilisée comme image de test.

L'extraction des caractéristiques est maintenant unifiée avec le module `features.py`. Le descripteur `cocr_grad` utilisé pendant l'entraînement est donc exactement le même que celui visualisé dans la section précédente, ce qui garantit la cohérence entre l'analyse exploratoire et la construction du modèle probabiliste. Pour chaque pixel, le vecteur d'observation est formé par (Cb , Cr , $|\nabla Y|$), c'est-à-dire deux composantes de chrominance et une mesure locale de contour.

Le flux d'annotation supervisée a également été étendu à plusieurs images. Chaque image est ouverte successivement, les régions d'intérêt annotées `skin` et `non_skin` sont converties en lots de pixels, puis toutes les annotations sont concaténées dans une base d'apprentissage unique avant l'ajustement du classifieur. Les commandes manuelles restent inchangées : `p` pour annoter la peau, `n` pour le non-peau, `r` pour réinitialiser la RoI courante et `q` pour passer à l'image suivante, ou terminer l'annotation lorsqu'il s'agit de la dernière image. Enfin, les métadonnées sauvegardées enregistrent désormais la liste des images d'apprentissage utilisées pour la

corrida.

Pour le protocole recommandé dans ce rapport, nous suggérons d'annoter `img/Essex_Faces/94/asamma.19.jpg` comme image d'apprentissage, puis de tester le modèle appris sur `img/Essex_Faces/94/ajones.19.jpg`, `img/Essex_Faces/94/asheal.18.jpg` et `img/Essex_Faces/96/acatsa.17.jpg`. Un jeu d'annotations raisonnable consiste à sélectionner environ 5 à 8 RoI de peau et 5 à 8 RoI de non-peau, en veillant à inclure parmi les négatifs le fond, les cheveux, les yeux, les sourcils et les zones d'ombre. Ce choix limite le risque que la composante de gradient favorise à tort des contours marqués qui ne correspondent pas à la peau.

5) **Résultats du modèle bayésien gaussien multidimensionnel:** Le cas A constitue un premier quasi-succès du modèle appris sur `asamma.19`. Le masque binaire localise correctement une grande partie de la peau du visage et l'*overlay* confirme que la région faciale principale est bien détectée. La carte de probabilité reste élevée sur les joues, le front et le menton, ce qui montre que le modèle reconnaît de manière cohérente les zones cutanées dominantes. On observe toutefois des faux positifs sur une partie du vêtement et autour de certaines transitions à fort contraste, ce qui indique que la séparation peau / non-peau n'est pas encore complète.

Le cas B présente lui aussi un résultat globalement satisfaisant, avec une bonne détection de la peau sur le visage et une structure générale assez proche de celle obtenue sur l'image d'apprentissage. Le masque et l'*overlay* montrent cependant que les cheveux sont partiellement classés comme peau, en particulier dans les zones claires et très contrastées. La carte de probabilité confirme que le modèle attribue localement une forte confiance à ces régions, ce qui suggère un recouvrement partiel entre certaines signatures de chrominance / gradient du visage appris et celles des cheveux dans cette image.

Le cas C est nettement moins convaincant. Le masque envahit une partie importante du fond et de plusieurs régions non cutanées, tandis que l'*overlay* montre une segmentation trop étendue autour du visage. La carte de probabilité attribue aussi des valeurs élevées à des zones qui ne devraient pas être interprétées comme de la peau. Le modèle parvient encore à repérer une partie du visage, mais il perd ici en spécificité et confond plus largement la peau avec le décor et certains objets de l'image.

Les cas A et B peuvent donc être considérés comme des cas de quasi-réussite. Dans les deux situations, le visage et la peau sont détectés de manière visiblement meilleure que le fond général, ce qui montre que le descripteur `cbr_grad` apporte une information utile. La combinaison de Cb , Cr et $|\nabla Y|$ préserve en effet une cohérence suffisante sur les zones principales de peau tout en exploitant une information locale de contour. Les faux positifs sur les vêtements dans le cas A ou sur les cheveux dans le cas B n'annulent pas ce résultat, mais ils révèlent que la frontière statistique apprise entre peau et non-peau reste imparfaite.

Ces erreurs s'expliquent en grande partie par le protocole d'apprentissage lui-même. Le modèle a été appris à partir d'une seule image, `asamma.19`, ce qui lui donne une vision très limitée de la variabilité réelle des apparences de peau et de non-peau. En outre, l'annotation manuelle n'a été réalisée qu'une seule fois ; elle peut donc être sensible à des RoI imparfaites, à un biais dans le choix des exemples et à une couverture incomplète des cas négatifs difficiles. Dans ce contexte, des régions comme les cheveux clairs, les vêtements ou certaines zones très contrastées peuvent partager avec la peau des signatures locales proches en chrominance ou en contraste, et `cbr_grad`, bien qu'il améliore la robustesse vis-à-vis de l'illumination, ne supprime pas totalement ce recouvrement statistique.

L'introduction de `cbr_grad` permet de mieux comprendre cette limite. L'introduction de `cbr_grad` réduit effectivement une partie de la sensibilité à la luminance seule, mais elle n'élimine pas le problème de changement de domaine entre l'image d'apprentissage et une image de test très différente. Ici, le fond, le style photographique, le contraste global et la distribution locale des couleurs et des textures de `acatsa.17` s'écartent davantage de ceux de `asamma.19`. Or le modèle QDA apprend une distribution gaussienne corrélée à partir des exemples annotés ; lorsqu'il n'a vu qu'une seule image d'entraînement, il généralise mal dès que l'apparence du nouveau cas s'éloigne trop des statistiques initiales. Autrement dit, `cbr_grad` aide à mieux représenter la peau, mais il reste dépendant des statistiques locales observées pendant l'apprentissage. Si le fond, les cheveux ou les objets du cas C présentent des combinaisons de Cb , Cr et $|\nabla Y|$ peu ou mal couvertes dans les annotations initiales, des faux positifs apparaissent malgré l'enrichissement du descripteur.

En conclusion, ces résultats montrent que `cbr_grad` constitue bien une amélioration par rapport à une représentation purement colorimétrique, car il combine l'information de peau et l'information de contour. Néanmoins, une seule image d'apprentissage et une seule itération d'annotation restent insuffisantes pour obtenir une généralisation robuste sur des images plus variées. L'amélioration la plus naturelle ne serait donc pas d'abandonner ce descripteur, mais d'augmenter la diversité des images d'apprentissage et de couvrir davantage de cas négatifs difficiles pendant l'annotation.

6) **Résultats du modèle bayésien gaussien naïve:** Dans le cas A, le modèle gaussien naïf fournit une détection qui reste cohérente sur l'essentiel du visage, tout en réduisant certains faux positifs observés précédemment sur les vêtements et autour de quelques bordures non faciales. Le masque est plus contenu, l'*overlay* apparaît plus propre sur les régions non cutanées, et la carte de probabilité reste concentrée sur les zones du visage les plus plausibles. L'explication la plus probable est que `GaussianNB` suppose une indépendance entre les composantes observées (Cb , Cr , $|\nabla Y|$), au lieu de modéliser leur covariance complète comme QDA. Avec une seule image d'entraînement, cette hypothèse plus simple peut rendre le modèle moins sensible à des corrélations spécifiques et accidentelles apprises sur `asamma.19`, ce qui le rend ici



Fig. 6. Cas A – ajones.19.jpg. De gauche à droite : image originale, masque binaire prédit, superposition de la détection sur l'image et carte de probabilité de peau.

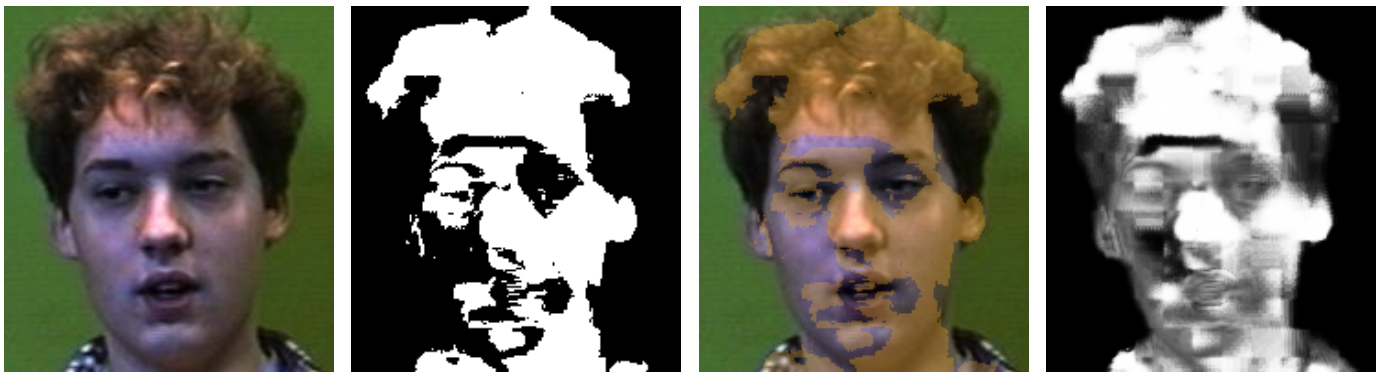


Fig. 7. Cas B – asheal.18.jpg. De gauche à droite : image originale, masque binaire prédit, superposition de la détection sur l'image et carte de probabilité de peau.



Fig. 8. Cas C – acatsa.17.jpg. De gauche à droite : image originale, masque binaire prédit, superposition de la détection sur l'image et carte de probabilité de peau.

plus conservateur hors de la région faciale principale.

Dans le cas B, le résultat obtenu avec le modèle naïf reste globalement très proche de celui du modèle gaussien multidimensionnel. La peau principale est toujours correctement détectée sur le visage, mais les faux positifs sur les cheveux demeurent du même ordre de grandeur. Cela suggère que, pour cette image, la difficulté ne provient pas d'abord de la manière de modéliser les corrélations entre composantes, mais plutôt du fait que certaines zones de cheveux possèdent déjà, au niveau même des caractéristiques locales C_b , C_r et $|\nabla Y|$, une signature assez proche de celle des zones de peau

appries. Dans un tel cas, passer d'une covariance complète à une hypothèse d'indépendance ne modifie pas fortement la décision finale.

Dans le cas C, le modèle naïf donne un résultat nettement meilleur que le modèle gaussien multidimensionnel, en particulier parce que le fond n'est plus massivement interprété comme de la peau. Le masque est plus resserré autour du visage, l'overlay laisse davantage le décor intact, et la carte de probabilité ne diffuse plus aussi fortement sur l'arrière-plan. Cette amélioration est remarquable car l'image originale présente justement un fond très différent de celui de



Fig. 9. Cas A-naïf – *ajones.19.jpg*. De gauche à droite : image originale, masque binaire prédit, superposition de la détection sur l'image et carte de probabilité de peau.



Fig. 10. Cas B-naïf – *asheal.18.jpg*. De gauche à droite : image originale, masque binaire prédit, superposition de la détection sur l'image et carte de probabilité de peau.



Fig. 11. Cas C-naïf – *acatsa.17.jpg*. De gauche à droite : image originale, masque binaire prédit, superposition de la détection sur l'image et carte de probabilité de peau.

asamma.19, qui a servi à l'entraînement. Le descripteur *cbr_grad* aide toujours à réduire la sensibilité à la luminosité seule, mais ici l'effet favorable vient surtout du fait que le modèle naïf généralise mieux sous changement de domaine : en n'ajustant pas les corrélations entre composantes à partir d'une image unique, il évite de sur-spécialiser sa frontière de décision aux statistiques particulières du cas d'apprentissage. L'hypothèse d'indépendance joue alors comme une forme de régularisation implicite.

Il est important de préciser que le caractère "naïf" du modèle ne signifie pas une absence de corrélation entre les

pixels de l'image. L'hypothèse porte sur l'indépendance conditionnelle des composantes du vecteur d'observation, c'est-à-dire ici Cb , Cr et $|\nabla Y|$, à l'intérieur d'une même classe. Le modèle gaussien multidimensionnel, au contraire, prend explicitement en compte les corrélations entre ces composantes.

En comparant les deux approches, le modèle gaussien naïf présente plusieurs avantages : il est plus simple, il semble moins sensible à un surapprentissage de corrélations accidentelles, et il se montre plus robuste dans certains cas de changement de domaine comme *acatsa.17*. Il peut aussi réduire certains faux positifs, comme sur les vêtements dans

le cas A. En revanche, il perd l'information sur les corrélations réelles entre Cb , Cr et $|\nabla Y|$, ce qui peut limiter sa précision lorsque ces dépendances sont effectivement discriminantes. On en conclut qu'aucun des deux modèles ne domine systématiquement l'autre : avec une seule image d'apprentissage, le modèle naïf peut mieux généraliser, tandis que le modèle gaussien multidimensionnel peut mieux exploiter la structure des données lorsque les statistiques de test restent proches de celles observées à l'entraînement.

B. Classification non supervisée des pixels par K-Means

Par rapport à l'approche bayésienne supervisée étudiée précédemment, K-Means constitue une alternative non supervisée intéressante pour la segmentation de la peau. Son principal avantage est qu'il ne repose pas sur une annotation manuelle préalable des pixels : il réduit donc la charge de supervision et limite les erreurs humaines liées au choix des RoI d'apprentissage. En contrepartie, les groupes obtenus par clustering ne correspondent pas nécessairement à des classes sémantiques directement exploitables comme *peau* et *non-peau*. Selon l'espace d'observation choisi, l'algorithme peut aussi produire des partitions davantage liées à l'illumination, au fond ou aux contrastes locaux qu'à la structure du visage elle-même. C'est pourquoi une phase d'expérimentation est nécessaire, afin d'identifier l'espace d'observation et le nombre de clusters les plus pertinents avant de tirer des conclusions sur l'intérêt pratique de cette approche.

1) **Protocole expérimental:** Le script `KMeans_Clustering.py` a d'abord été une démonstration minimale du clustering sur une image unique en espace BGR. Il a ensuite été adapté pour devenir un outil d'expérimentation reproductible, mieux aligné sur les besoins du TP. L'interface accepte maintenant une image d'apprentissage, une image de test optionnelle, le choix de l'espace d'observation, le nombre de classes K , la sélection manuelle des clusters interprétés comme peau, un répertoire de sortie et un mode sans affichage. Le point de départ retenu dans le script est `ycrcb` avec $K = 6$, mais ce choix doit justement être validé par les expériences.

L'extraction des caractéristiques a été unifiée avec le module `features.py`, ce qui permet de comparer dans un même cadre les espaces `rgb`, `hsv`, `ycrcb` et le descripteur local `cbr_grad`. Cette unification garantit que les vecteurs d'observation manipulés par K-Means sont construits de manière cohérente avec les autres scripts du projet. Dans le cas particulier de `cbr_grad`, une normalisation interne fondée uniquement sur l'image d'apprentissage est appliquée avant le clustering et la prédiction, afin d'éviter que la composante $|\nabla Y|$, de plus grande amplitude, ne domine artificiellement la distance euclidienne.

Le flux de calcul a lui aussi été étendu. Le modèle K-Means est appris sur l'image `train`, puis les mêmes centres de clusters sont réutilisés pour prédire les labels sur l'image `test`, ce qui rend la comparaison inter-images cohérente. Les sorties sont sauvegardées automatiquement dans un répertoire de la forme `outputs/kmeans/<test_stem>/`

puis `<features>_k<k>_train_<train_stem>/`. On y retrouve notamment `train_labels.png`, `test_labels.png`, `centers.json` et `metadata.json`. Lorsque les identifiants des clusters de peau sont fournis, le script enregistre aussi les masques binaires et les superpositions correspondantes pour l'image d'apprentissage et l'image de test.

Le protocole expérimental suit donc deux passes complémentaires. Dans un premier temps, on observe les cartes de labels et les centres de clusters afin d'identifier quels groupes semblent correspondre à la peau. Dans un second temps, on relance le script en indiquant manuellement les identifiants de clusters retenus pour produire les masques de peau et comparer les résultats selon l'espace d'observation et le nombre de classes. Ces étapes sont présentées dans la table I.

Ce protocole permet de séparer clairement trois questions. La première est le choix du meilleur espace d'observation pour isoler la peau sans supervision. La deuxième est l'influence du nombre de classes sur la fragmentation ou, au contraire, la fusion excessive des régions de peau et de non-peau. La troisième est la capacité du modèle appris sur `asamma.19` à conserver une interprétation cohérente des clusters lorsqu'il est appliqué à `ajones.19` puis à `acatsa.17`, deux cas de généralisation de difficulté différente.

2) **Comparaison initiale:** La comparaison initiale a été réalisée avec `train = test = asamma.19` et $K = 6$, afin d'évaluer la qualité intrinsèque des regroupements obtenus sur une image unique avant toute généralisation. Pour chaque espace d'observation, les clusters de peau ont été choisis manuellement à partir de `train_labels.png` et `centers.json`. Les groupes retenus sont les suivants : `rgb` avec les clusters $\{2, 3, 5\}$, `hsv` avec $\{0, 1, 3\}$, `ycrcb` avec $\{1, 3, 4\}$, et `cbr_grad` avec $\{2, 3\}$. Cette première étape sert donc à comparer la lisibilité des partitions et à estimer si les zones de peau forment ou non des groupes suffisamment cohérents pour être exploités dans la suite.

Le cas RGB est surprenamment bon et reste relativement proche du comportement obtenu avec YCrCb, même si la détection est un peu moins nette dans les détails fins. Sur cette image particulière, le contraste global entre le fond vert, les cheveux sombres et les zones de peau suffit déjà à produire des groupes utiles dans l'espace (R, G, B) . En revanche, RGB mélange directement luminance et chrominance, ce qui rend les clusters plus sensibles aux ombres et aux variations d'éclairage à l'intérieur du visage. Les bords des cheveux et les boucles d'oreilles sont ainsi parfois absorbés dans la peau, car leurs couleurs locales ou certains pixels de transition restent proches de celles de régions cutanées dans cet espace. Un raffinement peut passer par un K plus élevé pour séparer certaines zones ambiguës, mais aussi par des descripteurs plus discriminants, car la seule distance euclidienne de K-Means ne garantit pas une séparation sémantique parfaite.

Le cas HSV est le moins convaincant des quatre. Le masque est très bruité et fait apparaître plusieurs petites zones isolées classées comme peau alors qu'elles ne correspondent pas à des régions cutanées cohérentes. L'explication la plus probable

TABLE I
PROTOCOLE EXPÉRIMENTAL PROPOSÉ POUR L'ÉVALUATION DE K-MEANS. LA PREMIÈRE ÉTAPE SERT À CHOISIR LE MEILLEUR ESPACE D'OBSERVATION, LA DEUXIÈME À AJUSTER LE NOMBRE DE CLASSES K , ET LA TROISIÈME À ÉVALUER LA CAPACITÉ DE GÉNÉRALISATION SUR DES VISAGES NON UTILISÉS POUR L'APPRENTISSAGE.

Phase	Train	Test	Espace d'observation	K	Objectif
Comparaison initiale	asamma.19	asamma.19	rgb	6	Comparer la séparabilité obtenue avec RGB.
Comparaison initiale	asamma.19	asamma.19	hsv	6	Comparer la séparabilité obtenue avec HSV.
Comparaison initiale	asamma.19	asamma.19	ycrcb	6	Comparer la séparabilité obtenue avec YCrCb.
Comparaison initiale	asamma.19	asamma.19	cbcr_grad	6	Comparer la séparabilité obtenue avec le descripteur local enrichi.
Choix de K	asamma.19	asamma.19	meilleur espace	4, 6, 8	Sélectionner le nombre de clusters le plus pertinent.
Généralisation	asamma.19	ajones.19	meilleur espace	meilleur K	Tester un cas proche de l'image d'apprentissage.
Généralisation	asamma.19	acatsa.17	meilleur espace	meilleur K	Tester un cas plus différent et évaluer la robustesse.



Fig. 12. Cas RGB sur asamma.19.jpg. De gauche à droite : image originale, labels K-Means, masque des clusters interprétés comme peau et superposition finale.



Fig. 13. Cas HSV sur asamma.19.jpg. De gauche à droite : image originale, labels K-Means, masque des clusters interprétés comme peau et superposition finale.

est que la composante H devient instable dans les zones sombres ou faiblement saturées, exactement comme on l'avait déjà observé pendant l'analyse exploratoire des composantes. De plus, la distance euclidienne appliquée directement sur (H, S, V) n'est pas idéale, car la teinte est une variable circulaire et non une grandeur linéaire classique. Cette combinaison conduit K-Means à fragmenter davantage les régions de peau et à introduire des faux positifs dispersés, ce qui explique le caractère très bruité du résultat.

Le cas YCrCb apparaît comme le meilleur de cette comparaison initiale. La peau y est mieux distinguée, avec une segmentation plus homogène et davantage de détail sur le visage. Cette qualité s'explique par la séparation explicite entre la luminance Y et les composantes de chrominance Cr et

Cb , ce qui permet à K-Means de mieux regrouper les pixels cutanés malgré les variations d'éclairage internes. Le visage reste ainsi plus cohérent tout en conservant des structures fines plus lisibles. On observe encore quelques confusions au niveau du bord des cheveux et des boucles d'oreilles, car K-Means ne tient compte ni du contexte spatial ni d'une connaissance sémantique de l'objet à segmenter : il ne voit que la proximité des pixels dans l'espace d'observation. Un raffinement pourra donc passer par un K plus élevé, mais aussi par des descripteurs complémentaires ou par un post-traitement spatial.

Le descripteur `cbcr_grad`, pourtant prometteur dans le cadre bayésien supervisé, est ici moins convaincant avec K-Means. Le masque n'est pas uniforme : certaines zones de



Fig. 14. Cas YCrCb sur `asamma.19.jpg`. De gauche à droite : image originale, labels K-Means, masque des clusters interprétés comme peau et superposition finale.



Fig. 15. Cas `cbcr_grad` sur `asamma.19.jpg`. De gauche à droite : image originale, labels K-Means, masque des clusters interprétés comme peau et superposition finale.

peau ne sont pas reconnues comme telles, et de petits trous de non-peau apparaissent à l'intérieur même du visage. La raison la plus probable est que le gradient, même normalisé, introduit une information de structure locale qui tend à scinder la peau selon l'intensité des contours, des ombres et des transitions internes, plutôt que de regrouper toutes les zones cutanées sur la seule base de leur chrominance. Dans un modèle bayésien supervisé, cette information peut être exploitée positivement parce que l'apprentissage apprend quelles variations de gradient restent compatibles avec la peau. En revanche, dans un clustering non supervisé, K-Means ne dispose d'aucune information de classe et fragmente plus facilement la peau lorsqu'elle présente des variations locales de contour ou d'éclairement.

En synthèse, les quatre essais montrent que tous les espaces d'observation ne sont pas également adaptés à une segmentation non supervisée de la peau. HSV est clairement le plus bruyé, `cbcr_grad` fragmente trop la peau dans ce cadre non supervisé, et RGB donne un résultat déjà très honorable mais un peu moins fin et moins stable que YCrCb. Le meilleur compromis est obtenu avec YCrCb, qui préserve mieux l'homogénéité du visage, conserve davantage de détail utile et limite les faux positifs à des zones localement ambiguës. Pour cette raison, YCrCb sera retenu comme meilleur espace d'observation pour les prochaines étapes de l'expérimentation

K-Means, avant d'étudier plus finement l'effet du nombre de classes K .

REFERENCES

- [1] ONERA and ENSTA ParisTech, "Estimation et Identification statistiques — Cours 2 : Estimateur Bayésien," course slides (PDF), Sept. 13, 2023, accessed Feb. 18, 2026. [Course material]. Provided by the user.
- [2] A. Yuille, "Lecture 2. Bayes Decision Theory," course Stat 161/261, Johns Hopkins University, Spring 2014, accessed Feb. 18, 2026. [Online]. Available: <https://www.cs.jhu.edu/~ayuille1/courses/Stat161-261-Spring14/RevisedLectureNotes2.pdf>
- [3] OpenCV, "Color Space Conversions," OpenCV Documentation, version 3.4, accessed Mar. 7, 2026. [Online]. Available: https://docs.opencv.org/3.4/d8/d01/group_imgproc_color_conversions.html
- [4] A. R. Smith, "Color Gamut Transform Pairs," in *SIGGRAPH '78 Conference Proceedings*, Aug. 1978, pp. 12–19, accessed Mar. 7, 2026. [Online]. Available: <https://alvyray.com/Papers/CG/color78.pdf>
- [5] OpenCV, "Color conversions," OpenCV Documentation, version 3.4.20-dev, accessed Mar. 7, 2026. [Online]. Available: https://docs.opencv.org/3.4/de/d25/imgproc_color_conversions.html
- [6] OpenCV, "Color conversions," OpenCV Documentation, version 3.4.20-dev, accessed Mar. 7, 2026. [Online]. Available: https://docs.opencv.org/3.4/de/d25/imgproc_color_conversions.html